

Trabajo Práctico N° 2

Persistencia de Objetos con DAO + JDBC + MySQL

Proyecto Integrador – Sistema de Gestión de Turnos

1. Introducción

El presente trabajo práctico constituye la segunda etapa del proyecto integrador iniciado en el TP1.

En la primera etapa se trabajó sobre:

- Modelado orientado a objetos.
- Construcción de clases y relaciones.
- Encapsulamiento.
- Colecciones en Java.
- Maven.
- Pruebas unitarias con JUnit 5.

En esta nueva etapa se incorporará persistencia de datos utilizando:

- JDBC.
- MySQL.
- Patrón DAO.

El objetivo principal será persistir el modelo previamente construido manteniendo la separación entre:

- lógica del negocio,
 - modelo de objetos,
 - acceso a datos.
-

2. Objetivos

Al finalizar el trabajo práctico los estudiantes deberán ser capaces de:

- Comprender el funcionamiento básico de JDBC.
 - Conectarse desde Java a una base de datos MySQL.
 - Aplicar el patrón DAO.
 - Implementar operaciones CRUD.
 - Persistir objetos Java en tablas relacionales.
 - Relacionar el modelo UML con el modelo relacional.
 - Gestionar dependencias Maven para trabajar con MySQL.
-

3. Requisitos generales

El trabajo deberá realizarse reutilizando el proyecto desarrollado en el TP1.

No se deberá crear un proyecto nuevo desde cero.

Se deberá:

- mantener el modelo orientado a objetos ya construido,
- reorganizar paquetes si fuera necesario,
- incorporar persistencia mediante JDBC.

4. Estructura sugerida del proyecto

```
src/main/java/com/universitas
|
├── modelo
|   ├── Persona.java
|   ├── Paciente.java
|   ├── Profesional.java
|   ├── Especialidad.java
|   ├── Consultorio.java
|   ├── Turno.java
|   └── EstadoTurno.java
|
├── dao
|   ├── ConexionBD.java
|   ├── PacienteDAO.java
|   ├── ProfesionalDAO.java
|   ├── EspecialidadDAO.java
|   ├── ConsultorioDAO.java
|   └── TurnoDAO.java
|
└── app
    └── App.java
```

5. Base de datos

Nombre sugerido

```
CREATE DATABASE gestion_turnos;
```

6. Tablas mínimas requeridas

Tabla ESPECIALIDAD

```
CREATE TABLE especialidad (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100),  
  descripcion VARCHAR(255)  
);
```

Tabla PROFESIONAL

```
CREATE TABLE profesional (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100),  
  apellido VARCHAR(100),  
  dni VARCHAR(20),  
  matricula VARCHAR(50),  
  especialidad_id INT,  
  
  FOREIGN KEY (especialidad_id)  
    REFERENCES especialidad(id)  
);
```

Tabla PACIENTE

```
CREATE TABLE paciente (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nombre VARCHAR(100),  
  apellido VARCHAR(100),  
  dni VARCHAR(20),  
  numero_afiliado VARCHAR(50)  
);
```

Tabla CONSULTORIO

```
CREATE TABLE consultorio (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  numero INT,
```

```
sector VARCHAR(100)
);
```

Tabla TURNO

```
CREATE TABLE turno (
  id INT AUTO_INCREMENT PRIMARY KEY,

  paciente_id INT,
  profesional_id INT,
  consultorio_id INT,

  fecha DATE,
  hora TIME,
  estado VARCHAR(30),

  FOREIGN KEY (paciente_id)
    REFERENCES paciente(id),

  FOREIGN KEY (profesional_id)
    REFERENCES profesional(id),

  FOREIGN KEY (consultorio_id)
    REFERENCES consultorio(id)
);
```

7. Conceptos a aplicar

Durante el trabajo práctico deberán aplicarse los siguientes conceptos:

JDBC

- Connection
- DriverManager
- PreparedStatement
- ResultSet

SQL

- INSERT
- SELECT
- UPDATE
- DELETE

Modelo relacional

- PRIMARY KEY
- FOREIGN KEY
- Relaciones entre tablas

Programación orientada a objetos

- reutilización del modelo,
 - encapsulamiento,
 - separación de responsabilidades.
-

8. Patrón DAO

El acceso a datos deberá implementarse mediante clases DAO.

Objetivo

Separar:

- lógica de negocio,
- persistencia,
- modelo de objetos.

Las clases del modelo NO deberán contener instrucciones SQL.

9. Clase de conexión

Se deberá crear una clase encargada de centralizar la conexión con la base de datos.

Ejemplo orientativo

```
package com.universitas.dao;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConexionBD {

    private static final String URL =
        "jdbc:mysql://localhost:3306/gestion_turnos";

    private static final String USER = "root";
    private static final String PASSWORD = "root";
```

```
public static Connection obtenerConexion() throws SQLException {  
    return DriverManager.getConnection(URL, USER, PASSWORD);  
}  
}
```

10. DAOs mínimos requeridos

PacienteDAO

Operaciones mínimas:

- guardar
- listar

ProfesionalDAO

Operaciones mínimas:

- guardar
- listar

TurnoDAO

Operaciones mínimas:

- guardar
- listar
- buscarPorId
- eliminar

11. Ejemplo orientativo de método DAO

```
public void guardar(Paciente paciente) {  
  
    String sql = ""  
        INSERT INTO paciente  
        (nombre, apellido, dni, numero_afiliado)  
        VALUES (?, ?, ?, ?)  
        "";  
  
    try (
```

```

        Connection conn = ConexionBD.obtenerConexion();
        PreparedStatement ps = conn.prepareStatement(sql)
    ) {

        ps.setString(1, paciente.getNombre());
        ps.setString(2, paciente.getApellido());
        ps.setString(3, paciente.getDni());
        ps.setString(4, paciente.getNumeroAfiliado());

        ps.executeUpdate();

        System.out.println("Paciente guardado correctamente.");

    } catch (SQLException e) {
        e.printStackTrace();
    }
}

```

12. Requisitos de Maven

El archivo pom.xml deberá incluir:

- Configuración del JDK utilizado.
- Dependencia MySQL Connector/J.
- Dependencia JUnit 5.
- Plugin Surefire.
- Configuración para generar JAR ejecutable.

13. Dependencia MySQL requerida

```

<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>8.3.0</version>
</dependency>

```

14. Requisitos funcionales

El sistema deberá permitir:

- registrar pacientes,
- registrar profesionales,

- registrar especialidades,
- registrar consultorios,
- agendar turnos,
- listar turnos,
- buscar turnos,
- eliminar turnos.

15. Requisitos de testing

Se deberán mantener y ampliar los tests unitarios desarrollados en el TP1.

Como mínimo deberán verificarse:

- constructores,
- getters,
- setters,
- métodos de negocio,
- operaciones básicas sobre colecciones.

16. Requisitos de entrega

La entrega deberá incluir:

- Código fuente completo.
- Base de datos creada en MySQL.
- Script SQL de creación de tablas.
- Proyecto Maven funcional.
- DAOs implementados.
- Evidencia de operaciones CRUD funcionando.
- pom.xml actualizado.

17. Criterios de evaluación

Aspecto	Descripción	Puntaje sugerido
Modelo reutilizado correctamente	Reutilización adecuada del modelo OO del TP1	15%
Diseño DAO	Correcta separación entre modelo y persistencia	20%
JDBC	Uso correcto de Connection, PreparedStatement y ResultSet	25%
SQL y relaciones	Diseño correcto de tablas y relaciones	15%

Aspecto	Descripción	Puntaje sugerido
Maven	Dependencias y configuración correcta del proyecto	10%
CRUD funcional	Persistencia y operaciones funcionando correctamente	15%

18. Recomendaciones

- Mantener el modelo orientado a objetos separado de la persistencia.
- Evitar escribir SQL dentro de las clases del modelo.
- Utilizar PreparedStatement para parametrizar consultas.
- Reutilizar la clase de conexión.
- Organizar el proyecto por paquetes.
- Mantener código prolijo y documentado.

19. Observación importante

Este trabajo práctico forma parte de una secuencia integradora.

En etapas posteriores el mismo sistema evolucionará hacia:

- aplicaciones web utilizando JSP y Servlet,
- patrón MVC,
- Spring Boot,
- arquitecturas por capas.

Por este motivo, es importante mantener:

- un modelo claro,
- responsabilidades separadas,
- organización adecuada del proyecto.